

Interview Handbook

Concise Edition for Intermediate SRE / Cloud Operations Engineers

Target: 4 Years AWS Experience | DevOps | SRE | Platform Engineer

Covers: Fundamentals · State · Modules · Lifecycle · AWS · CI/CD · Troubleshooting

Table of Contents

Module 1 — Terraform Fundamentals & Architecture

- Chapter 1 – What is Terraform?
- Chapter 2 – Terraform Workflow (init → plan → apply → destroy)
- Chapter 3 – Providers & Resources

Module 2 — Terraform Language

- Chapter 4 – Variables, Locals & Outputs
- Chapter 5 – Data Sources, Expressions & Key Functions

Module 3 — Terraform State ★★★★★

- Chapter 6 – State Deep Dive
- Chapter 7 – Remote State, S3 Backend & DynamoDB Locking
- Chapter 8 – State Commands, Drift & State Recovery

Module 4 — Modules ★★★★★

- Chapter 9 – Modules, Child Modules & Project Structure

Module 5 — Resource Lifecycle & Dependencies ★★★★★

- Chapter 10 – Lifecycle Meta Arguments & depends_on
- Chapter 11 – count & for_each

Module 6 — AWS + Terraform ★★★★★

- Chapter 12 – AWS Integration (IAM, VPC, EC2, ALB, ASG, S3, SSM, KMS)

Module 7 — Production, CI/CD & Troubleshooting ★★★★★

- Chapter 13 – CI/CD Integration
- Chapter 14 – 25 Production Troubleshooting Scenarios

Module 8 — Interview Revision

- Chapter 15 – Top 80 Interview Questions
- Chapter 16 – Cheat Sheets & One-Day Revision Guide

Chapter 1 – What is Terraform?

Definition: Terraform is an open-source Infrastructure as Code (IaC) tool by HashiCorp. It lets you define cloud infrastructure in declarative HCL (HashiCorp Configuration Language) files and provision it across providers like AWS, Azure, and GCP.

Why it exists: Before IaC, infrastructure was created manually through cloud consoles or scripts, making it hard to track, reproduce, or audit. Terraform solves this by treating infrastructure like software code — versioned, reviewable, and repeatable.

Core Design Principles

- **Declarative:** You describe the desired end-state; Terraform figures out how to reach it.
- **Idempotent:** Running the same config multiple times produces the same result.
- **Provider-agnostic:** Works with 3,000+ providers via a plugin system.
- **State-driven:** Tracks what's deployed in a state file and plans only what needs to change.

Terraform vs CloudFormation

Feature	Terraform	CloudFormation
Language	HCL (also JSON)	YAML / JSON
Multi-cloud	Yes	AWS only
State management	External state file	AWS-managed
Community modules	Huge registry	Limited
Drift detection	terraform plan	Drift detection (newer)
Open source	Yes (MPL 2.0)	No

★★★★★ Almost Every Interview

Q: What is Terraform and why would you choose it over CloudFormation?

A: Terraform is an open-source IaC tool that manages infrastructure across multiple providers using declarative HCL. I'd choose it over CloudFormation for multi-cloud flexibility, a rich community module ecosystem, and familiar workflow across AWS, GCP, and Azure.

CloudFormation is tightly integrated with AWS and managed by AWS, which has advantages in IAM integration and rollback, but Terraform offers more portability and reusability.

Chapter 2 – Terraform Workflow

Every Terraform deployment follows the same core workflow:

```
terraform init      → Download providers, initialize backend
terraform validate → Check HCL syntax and configuration
terraform fmt       → Auto-format code
terraform plan      → Preview changes (diff against state)
terraform apply     → Create/update/destroy resources
terraform destroy   → Tear down all managed resources
```

What happens internally during terraform plan

- Terraform reads all .tf files and builds the configuration.
- Loads state file from local or remote backend.
- Calls provider APIs to refresh current resource state.
- Computes the dependency graph (DAG — Directed Acyclic Graph).
- Compares desired state vs current state and outputs a diff.
- No changes are made to infrastructure during plan.

What happens during terraform apply

- Runs a fresh plan internally.
- Prompts for confirmation (or uses -auto-approve).
- Executes resource creates, updates, and destroys in dependency order.
- After each resource action, updates the state file.
- Stops on error; partial state is saved.

★★★★★ Frequently Asked

Q: What happens if terraform apply is interrupted midway?

A: Terraform writes state incrementally. Resources created before the interruption are recorded in state; resources not yet created are not. On next apply, Terraform will attempt to create remaining resources. Manually verify state with 'terraform state list' and check AWS console for partial resources. A DynamoDB lock may need to be released if using remote state.

★★★★★ Almost Every Interview

Q: Difference between terraform plan and terraform apply?

A: Plan is a dry-run that shows what Terraform would change without making any modifications. Apply executes those changes. Always run plan first in production, review it, then apply. In CI/CD, save the plan file and apply exactly that plan to avoid drift between review and execution.

★★★★■ Frequently Asked

Q: What does terraform init do?

A: It initializes the working directory: downloads provider plugins, sets up the backend (local or remote), and installs modules. Must be run before plan or apply. Safe to run multiple times — it is idempotent.

Chapter 3 – Providers & Resources

Providers are plugins that translate Terraform configuration into API calls for a specific platform (AWS, Azure, GCP, Kubernetes, etc.). Each resource type belongs to a provider.

```
terraform {
  required_providers {
```

```

    aws = {
      source = "hashicorp/aws"
      version = "~> 5.0"      # pin to major version
    }
  }
}

provider "aws" {
  region = "us-east-1"
}

```

Version Constraints

Operator	Meaning	Example
=	Exact version	= 5.0.0
~>	Allow patch/minor increments	~> 5.0 (5.x.x)
>=	Minimum version	>= 5.0
<=	Maximum version	<= 5.5

Always pin provider versions in production. The `.terraform.lock.hcl` file locks exact versions after init — commit this file to git.

Resources are the building blocks — individual infrastructure objects declared with a type and name:

```

resource "aws_instance" "web" {
  ami           = "ami-0abcdef1234567890"
  instance_type = "t3.micro"
  tags = {
    Name        = "web-server"
    Environment = "prod"
  }
}

```

★★★★■ Frequently Asked

Q: What is the `.terraform.lock.hcl` file and should you commit it?

A: It records the exact provider versions and checksums resolved during terraform init. You should commit it to version control so every team member and CI/CD pipeline uses identical provider versions, preventing 'works on my machine' issues from version drift.

Chapter 4 – Variables, Locals & Outputs

Input Variables — parameterize configurations

```
variable "instance_type" {
  description = "EC2 instance type"
  type        = string
  default     = "t3.micro"
  validation {
    condition   = contains(["t3.micro", "t3.small", "t3.medium"], var.instance_type)
    error_message = "Must be a valid t3 type."
  }
}
```

Supply values via: `terraform.tfvars`, `-var` flag, environment variables (`TF_VAR_instance_type`), or interactive prompt.

Locals — computed, reusable values within a module

```
locals {
  env          = terraform.workspace
  name_prefix = "${var.project}-${local.env}"
  common_tags = {
    Project      = var.project
    Environment  = local.env
    ManagedBy    = "Terraform"
  }
}
```

Use locals to avoid repeating complex expressions and to centralize naming conventions.

Outputs — expose values to callers or display after apply

```
output "instance_public_ip" {
  value       = aws_instance.web.public_ip
  description = "Public IP of the web server"
  sensitive   = false
}
```

In modules, outputs are how child modules pass data back to the root module or to other modules via the root.

★★★★■ Frequently Asked

Q: What is the difference between a variable and a local?

A: Variables are inputs — they can be set externally by users or CI/CD. Locals are computed values defined within the module and cannot be overridden from outside. Use locals to derive values from variables (e.g., name prefixes, common tags) to avoid repeating logic.

★★★ Sometimes Asked

Q: How do you mark an output as sensitive?

A: Set 'sensitive = true' on the output block. Terraform will mask the value in CLI output and plan but still stores it in state. Use this for passwords, tokens, and secret ARNs.

Chapter 5 – Data Sources, Expressions & Key Functions

Data Sources let you read existing infrastructure not managed by Terraform:

```
data "aws_ami" "amazon_linux" {
  most_recent = true
  owners      = ["amazon"]
  filter {
    name     = "name"
    values   = ["amzn2-ami-hvm-*x86_64-gp2"]
  }
}

resource "aws_instance" "web" {
  ami = data.aws_ami.amazon_linux.id # reference data source
}
```

Resource vs Data Source

	Resource	Data Source
Prefix	resource	data
Manages lifecycle?	Yes creates/updates/destroys	No — read-only
In state?	Yes	No (fetched at plan)
Use case	Create new infra	Reference existing infra

Key Functions (most asked in interviews)

- `length(list)` — count of items
- `toset(list)` — convert to set (used with `for_each`)
- `lookup(map, key, default)` — safely look up map key
- `merge(map1, map2)` — merge maps (used for tag merging)
- `format("fmt", args...)` — string interpolation
- `jsonencode(obj)` — encode to JSON string (IAM policies)
- `file(path)` — read file content
- `templatefile(path, vars)` — render template
- `coalesce(a, b, c)` — return first non-empty value

State is the most tested Terraform topic in SRE/DevOps interviews. Master every section here.

Chapter 6 – State Deep Dive

What is Terraform State?

The state file (terraform.tfstate) is a JSON file that maps your Terraform configuration to real-world infrastructure. It records resource IDs, attributes, dependencies, and metadata. Terraform uses state to:

- Know what resources it manages (vs. resources created outside Terraform).
- Build the dependency graph for planning.
- Detect drift between desired and actual state.
- Track resource metadata (e.g., ARNs, IPs) needed by dependent resources.

Local vs Remote State

	Local State	Remote State
Location	terraform.tfstate on disk	S3, Terraform Cloud, etc.
Team use	Not suitable — no locking	Yes — with locking
Security	Plain text on disk	Encrypted, access-controlled
CI/CD	Difficult	Required
Recommended for prod?	No	Yes

★★★★★ Almost Every Interview

Q: Why should you never use local state in production?

A: Local state is stored as plain text on the engineer's machine. In a team, multiple engineers can't share it; there's no locking so concurrent applies cause corruption; and CI/CD pipelines can't access it. In production always use remote state with a backend like S3 + DynamoDB for locking.

★★★★★ Almost Every Interview

Q: What happens if two engineers run terraform apply at the same time?

A: Without state locking, both reads would see the same current state and both would attempt to make changes, causing state corruption and resource conflicts. With DynamoDB locking, the second apply immediately sees a LockAcquireFailed error and waits or fails gracefully.

★★★★■ Frequently Asked

Q: What information is stored in the state file?

A: Resource type and name, provider configuration, all resource attributes (IDs, ARNs, IPs, names), dependency metadata, module structure, and a serial number that increments on each change. Sensitive values like passwords are stored in plain text — a key reason to encrypt state at rest.

Chapter 7 – Remote State, S3 Backend & DynamoDB Locking

S3 + DynamoDB Backend (AWS Standard)

```
terraform {  
  backend "s3" {  
    bucket      = "my-company-terraform-state"  
    key         = "production/app1/terraform.tfstate"  
    region      = "us-east-1"  
    encrypt     = true  
    kms_key_id  = "alias/terraform-state"  
    dynamodb_table = "terraform-state-lock"  
  }  
}
```

DynamoDB Locking — How it Works

- Terraform creates a row in DynamoDB with the lock ID, info, and timestamp on apply/plan start.
- If another process tries to apply and finds the row, it fails with LockAcquireFailed.
- After apply completes (success or failure), Terraform deletes the lock row.
- DynamoDB table must have a primary key named LockID (type String).

Production Best Practices for S3 Backend

- Enable S3 versioning — allows recovery of previous state versions.
- Enable server-side encryption (SSE-KMS).
- Block all public access on the S3 bucket.
- Use separate buckets per AWS account.
- Use a separate state file per environment and per application.
- Restrict S3/DynamoDB access with least-privilege IAM policies.

Remote State as Data Source

```
data "terraform_remote_state" "vpc" {  
  backend = "s3"  
  config = {  
    bucket = "my-company-terraform-state"  
    key    = "production/vpc/terraform.tfstate"  
    region = "us-east-1"  
  }  
}  
  
resource "aws_instance" "web" {
```

```
    subnet_id = data.terraform_remote_state.vpc.outputs.private_subnet_id
}
```

This pattern allows separate Terraform projects to share outputs (e.g., VPC IDs, subnet IDs) without duplicating configuration or coupling codebases.

★★★★★ Frequently Asked

Q: How do you share VPC IDs between separate Terraform projects?

A: The VPC project exposes the VPC ID as an output. Other projects read it via `terraform_remote_state` data source pointing at the VPC state file in S3. This keeps projects decoupled — the consuming project doesn't need the VPC code, only read access to the state bucket.

Chapter 8 – State Commands, Drift & State Recovery

Essential State Commands

```
terraform state list           # list all managed resources
terraform state show aws_instance.web    # inspect a specific resource
terraform state mv aws_instance.old aws_instance.new # rename/move resource in state
terraform state rm aws_s3_bucket.legacy  # remove from state (not from AWS)
terraform state pull           # download and print current state
terraform state push terraform.tfstate    # upload state (danger: use carefully)
```

Drift Detection

Drift occurs when real AWS infrastructure diverges from Terraform state — usually because someone made a manual change in the AWS console or via CLI.

- `terraform plan` — shows drift as proposed changes.
- `terraform refresh` — updates state from real infrastructure (deprecated as standalone; now integrated into plan).
- Resolution: re-apply Terraform to enforce desired state, or use `ignore_changes` for intentional external changes.

Disaster Recovery Scenarios

State file deleted	Restore from S3 versioning (<code>terraform state push</code>), or re-import all resources manually with <code>terraform import</code> .
State file corrupted	Restore previous version from S3 versioning. Use <code>terraform state pull/push</code> to replace.
DynamoDB lock stuck	Verify no apply is running. Delete the lock row from DynamoDB manually: <code>aws dynamodb delete-item --table-name terraform-state-lock --key '{"LockID": {"S": ""}}'</code> .
Accidental terraform destroy	Restore state from S3 backup. Investigate S3 versioning/logging. Re-apply to recreate resources. Implement <code>prevent_destroy</code> lifecycle.

State drift

Run terraform plan to see differences. Decide: re-apply Terraform or use ignore_changes. Document manual changes as tech debt.

★★★★★ Almost Every Interview

Q: How do you recover if the Terraform state file is accidentally deleted?

A: First check S3 versioning — restore the previous state file version. Then run terraform state push to upload it. If no backup exists, use terraform import to reimport each resource into a new state file — tedious but possible. Going forward: enable S3 versioning and restrict who can delete state files.

★★★★■ Frequently Asked

Q: When would you use terraform state rm?

A: When you want to stop managing a resource with Terraform without deleting it from AWS. For example, if a resource is being handed off to another team's Terraform project or being imported into a different state file. The resource stays in AWS; Terraform just forgets it.

Chapter 9 – Modules, Child Modules & Project Structure

What is a Module?

A module is a reusable collection of Terraform configuration files that performs a specific infrastructure task — think of it as a function in programming. Every Terraform configuration is technically a module (the root module). Child modules are reusable components called by the root.

Root vs Child Module

	Root Module	Child Module
Definition	Directory where you run terraform commands	Reusable component called by root
Applied directly?	Yes	No — called via module block
Contains	main.tf, variables.tf, outputs.tf	Same structure, but no backend config

Module Structure

```
modules/  
  vpc/  
    main.tf  
    variables.tf  
    outputs.tf  
    README.md  
  ec2/  
    main.tf  
    variables.tf  
    outputs.tf  
  
environments/  
  dev/  
    main.tf  
    terraform.tfvars  
    backend.tf  
  prod/  
    main.tf  
    terraform.tfvars  
    backend.tf
```

Calling a Module

```
module "vpc" {
```

```

source = "./modules/vpc"           # local
# OR
source = "terraform-aws-modules/vpc/aws" # registry
version = "5.1.2"                   # always pin!

cidr_block = "10.0.0.0/16"
environment = var.environment
}

# Access module output
resource "aws_instance" "web" {
  subnet_id = module.vpc.private_subnet_id
}

```

Module Communication Pattern

- Module A exposes data via **outputs**.
- Root module receives that output.
- Root module passes it as a variable to Module B.
- Modules do not reference each other directly — always via root.

Module Best Practices

- Always pin module versions (prevents breaking changes from registry updates).
- One module = one responsibility (VPC module, EC2 module, IAM module).
- Accept environment-specific values as input variables — never hardcode.
- Expose useful outputs so callers can integrate.
- Include a README and variable descriptions for maintainability.
- Store private modules in a Git registry or Terraform Cloud Private Registry.

★★★★★ Almost Every Interview

Q: How do modules communicate with each other?

A: They don't directly. Module A outputs a value (e.g., `vpc_id`). The root module captures that output (`module.vpc.vpc_id`) and passes it as an input variable to Module B. This keeps modules decoupled and testable independently.

★★★★■ Frequently Asked

Q: Why pin module versions?

A: The Terraform registry or a remote source can release breaking changes. Without pinning, 'terraform init' on a new machine or CI runner might pull a newer version that changes resource behavior, breaks your code, or destroys and recreates resources. Version pinning ensures reproducibility.

★★★★★ Architecture Interview

Q: Design a Terraform repository for a company with Dev, QA, and Prod environments.

A: Separate environments into directories under 'environments/', each with its own backend config and `.tfvars`. Place reusable infrastructure in 'modules/' (vpc, ec2, alb, iam). Each environment's `main.tf` calls the same modules but with different variable values. State files are separate per environment. CI/CD deploys environments independently with environment-specific approvals.



Chapter 10 – Lifecycle Meta Arguments & depends_on

depends_on — Explicit Dependencies

Terraform automatically infers dependencies through resource references (implicit). Use `depends_on` only when Terraform cannot detect the relationship:

```
resource "aws_instance" "web" {
  ami           = data.aws_ami.amazon_linux.id
  instance_type = "t3.micro"
  iam_instance_profile = aws_iam_instance_profile.web.name

  depends_on = [
    aws_iam_role_policy_attachment.web_ssm
  ]
}
```

The IAM role attachment isn't referenced in an attribute, so Terraform can't infer the dependency. `depends_on` forces correct ordering.

lifecycle Block Options

Option	Purpose	Common Use Case
<code>create_before_destroy</code>	Create replacement before destroying original	ALB, ASG, Launch Templates — minimize downtime
<code>prevent_destroy</code>	Block terraform destroy on this resource	Production RDS, S3 buckets, KMS keys
<code>ignore_changes</code>	Ignore specified attribute changes in future plans	Tags managed by external tools, AMI IDs
<code>replace_triggered_by</code>	Force replacement when another resource changes	Force ASG refresh when Launch Template changes

```
resource "aws_db_instance" "main" {
  identifier      = "prod-db"
  instance_class = "db.t3.medium"
  # ...

  lifecycle {
    prevent_destroy      = true
    ignore_changes       = [engine_version, snapshot_identifier]
    create_before_destroy = true
  }
}
```

```
}  
}
```

★★★★★ Almost Every Interview

Q: What is the default destroy behavior and how does `create_before_destroy` change it?

A: By default Terraform destroys the old resource first, then creates the new one — causing downtime. With `create_before_destroy=true`, Terraform creates the replacement first, then destroys the original. This requires that both resources can coexist (e.g., unique name constraints may prevent this without a name suffix strategy).

★★★★★ Frequently Asked

Q: A security team adds tags to all EC2 instances automatically. How do you prevent Terraform from removing those tags?

A: Use `'ignore_changes = [tags]'` in the lifecycle block. This tells Terraform to ignore tag drift for that resource while still managing all other attributes. Apply it only to specific attributes — avoid `'ignore_changes = all'` as it causes Terraform to miss real drift.

★★★ Interview Trap

Q: Does `depends_on` improve performance?

A: No — it can reduce performance. Terraform executes independent resources in parallel. `depends_on` adds a sequencing requirement, forcing resources to wait even when they could run concurrently. Only use it when a real undetectable dependency exists.

Chapter 11 – `count` & `for_each`

	<code>count</code>	<code>for_each</code>
Input type	Number	Map or Set of strings
Identifies resources by	Index (0, 1, 2...)	Key (string)
Removing middle item	Destroys and recreates downstream items	Only removes that specific item
Recommended for?	Simple N identical resources	Named or heterogeneous resources
Access syntax	<code>resource.name[0]</code>	<code>resource.name["key"]</code>

```
# count – creates 3 identical subnets  
resource "aws_subnet" "public" {  
  count          = 3  
  vpc_id         = aws_vpc.main.id  
  cidr_block     = cidrsubnet(var.vpc_cidr, 8, count.index)  
  availability_zone = data.aws_availability_zones.available.names[count.index]  
}
```

```
# for_each – creates subnets with meaningful names  
resource "aws_subnet" "public" {
```

```
for_each      = toset(["us-east-1a", "us-east-1b", "us-east-1c"])
vpc_id        = aws_vpc.main.id
cidr_block    = var.subnet_cidrs[each.key]
availability_zone = each.key
}
```

★★★★★ Almost Every Interview — Interview Trap

Q: You have 3 subnets managed with count. You need to remove the middle one. What happens?

A: With count, Terraform identifies resources by index. Removing index 1 (middle) causes Terraform to shift index 2 down to index 1, triggering a destroy and recreate of the resource that was at index 2. In production this can cause unintended outages. The fix is to migrate to `for_each` which uses string keys — deleting a key only affects that specific resource.

Chapter 12 – AWS Integration

IAM — Least Privilege for Terraform

- Terraform needs an IAM user or role with permissions to create/modify/destroy the resources it manages.
- In CI/CD use IAM Roles (not access keys) — attach roles to EC2 instances, ECS tasks, or use OIDC for GitHub Actions.
- Never store `AWS_ACCESS_KEY_ID` / `AWS_SECRET_ACCESS_KEY` in .tf files or git.
- Separate IAM roles per environment (dev, staging, prod) with escalating restrictions.

VPC with Terraform

```
module "vpc" {
  source = "terraform-aws-modules/vpc/aws"
  version = "5.1.2"

  name            = "prod-vpc"
  cidr            = "10.0.0.0/16"
  azs             = ["us-east-1a", "us-east-1b", "us-east-1c"]
  private_subnets = ["10.0.1.0/24", "10.0.2.0/24", "10.0.3.0/24"]
  public_subnets  = ["10.0.101.0/24", "10.0.102.0/24", "10.0.103.0/24"]

  enable_nat_gateway = true
  single_nat_gateway = false # HA: one per AZ

  tags = local.common_tags
}
```

EC2 + Auto Scaling

- Use **Launch Templates** (not launch configs) — supports versioning, Mixed Instances Policy.
- ASG with Launch Template: on Launch Template change, use `replace_triggered_by` lifecycle to force instance refresh.
- Use `instance_refresh` block in ASG for rolling replacement.
- Tag instances via Launch Template tags block for cost allocation and SSM inventory.

Secrets Management — Never in .tf Files

- **AWS Secrets Manager / SSM Parameter Store**: store secrets there; reference via data source.
- **Sensitive variables**: mark with `sensitive = true`; prevents display in logs but does NOT prevent storage in state file.
- **Environment variables**: pass secrets via `TF_VAR_*` env vars in CI/CD.
- **KMS**: encrypt state at rest; encrypt SSM SecureString parameters.

```
data "aws_secretsmanager_secret_version" "db_password" {
  secret_id = "prod/db/password"
}

resource "aws_db_instance" "main" {
  password = data.aws_secretsmanager_secret_version.db_password.secret_string
}
```

★★★★★ Frequently Asked

Q: How do you authenticate Terraform to AWS in CI/CD without access keys?

A: In GitHub Actions, configure OIDC trust between GitHub and AWS. The workflow assumes an IAM role using 'configure-aws-credentials' action with role-to-assume. No long-lived credentials are stored. For Jenkins on EC2, attach an IAM instance profile to the Jenkins server. This follows AWS security best practices — short-lived credentials via STS.

★★★★■ Interview Trap

Q: Sensitive variable is marked sensitive=true. Is it safe?

A: It's safer — Terraform hides the value in plan/apply output and logs. But the value is still stored in plain text in the state file. The state file must be encrypted (S3 SSE-KMS) and access must be restricted via IAM. sensitive=true is not encryption — it is only display masking.

Chapter 13 – CI/CD Integration

GitHub Actions — Terraform Pipeline

```
name: Terraform Deploy
on:
  push:
    branches: [main]
  pull_request:

permissions:
  id-token: write    # for OIDC
  contents: read

jobs:
  terraform:
    runs-on: ubuntu-latest
    steps:
      - uses: actions/checkout@v4
      - uses: aws-actions/configure-aws-credentials@v4
        with:
          role-to-assume: arn:aws:iam::123456789:role/terraform-ci
          aws-region: us-east-1
      - uses: hashicorp/setup-terraform@v3
        with:
          terraform_version: "1.7.0"
      - run: terraform init
      - run: terraform plan -out=tfplan
      - run: terraform apply tfplan # only on push to main
```

Best Practices for Terraform in CI/CD

- Run `terraform fmt -check` and `terraform validate` on every pull request.
- Save plan output with `-out=tfplan` and apply exactly that plan — prevents drift between plan and apply.
- Use separate pipelines per environment with manual approvals for production.
- Store state in S3 with DynamoDB locking — never local state in CI.
- Use OIDC instead of long-lived access keys.
- Run security scans (tfsec, checkov) in the pipeline before plan.
- Post plan output as a PR comment so reviewers can see changes.

Atlantis (GitOps Terraform)

Atlantis is a self-hosted pull request automation tool. It runs plan on every PR and applies on merge (or via 'atlantis apply' comment). Useful for teams wanting GitOps workflow without Terraform Cloud.

Chapter 14 – 25 Production Troubleshooting Scenarios

Scenario 1: State file deleted

Restore from S3 versioning. terraform state push restored.tfstate. If no backup: use terraform import for each resource. Enable versioning and restrict delete permissions going forward.

Scenario 2: DynamoDB lock not released

Confirm no apply is running (check CI/CD jobs). aws dynamodb delete-item with the LockID. Run terraform plan to verify state is accessible. Investigate why lock wasn't released — likely an interrupted apply.

Scenario 3: terraform plan shows unexpected destroy

Read plan carefully — identify which attribute change is forcing replacement. Check if a count → for_each migration is needed. Use lifecycle ignore_changes if the attribute is intentionally externally managed. Never apply a destroy plan without understanding the reason.

Scenario 4: Partial terraform apply failure

Resources created before failure are in state. Resources after failure are not in AWS but Terraform doesn't know. Run terraform plan to see remaining work. Investigate the root cause (IAM permission denied, quota exceeded, API throttling). Fix the issue then re-apply — Terraform is idempotent and will only create what's missing.

Scenario 5: AWS API throttling during apply

Use TF_LOG=DEBUG to see API call details. Add depends_on to reduce parallelism. Set -parallelism=5 (default 10) to reduce concurrent API calls. For large deployments, split into smaller apply operations. AWS throttle limits are per-service, per-region, per-account.

Scenario 6: Terraform plan shows drift after manual AWS change

terraform plan reveals the drift. Decide: re-apply Terraform to revert the change, or use ignore_changes if the manual change is intentional. Document manual changes in the incident report. Long-term: restrict console access; enforce all changes through Terraform.

Scenario 7: Provider version conflict

Error: providers selected in lock file are not compatible with this Terraform version. Run terraform init -upgrade to update lock file. Review changelog for breaking changes. Test in dev first. Pin to a specific working version in required_providers.

Scenario 8: Module version upgrade breaks deployment

Pin to previous version while investigating. Read the module changelog. If breaking change: update calling code, test in dev, then promote. Never blindly upgrade modules in production.

Scenario 9: terraform import fails

Error: resource already exists OR import doesn't populate all state fields. Check exact import syntax in provider docs. After import, run terraform plan — drift will show attributes not in config. Add them to config. Test import in dev first.

Scenario 10: Cross-account deployment fails

Terraform assumes role via provider alias or assume_role config. Check role trust policy allows the deploying account/role. Check permissions in target account. Use aws sts assume-role manually to verify trust. Check STS session duration limits.

Scenario 11: Secrets leaked into state file

Rotate the secret immediately. Restrict access to state bucket. Remove sensitive data from state using terraform state rm and re-import if needed. Enable state encryption. Mark variables as sensitive. Audit who has S3 read access to state bucket.

Scenario 12: Backend unavailable (S3 unreachable)

terraform init and plan will fail. Check S3 bucket exists, bucket policy, VPC endpoint if in private network. Check AWS service health. If truly unavailable: do not run apply. Wait for recovery. Consider cross-region S3 replication for state bucket resilience.

Scenario 13: Terraform apply destroys a production resource unexpectedly

Immediately check if prevent_destroy was missing. Restore from backup/snapshot if available. Use terraform import to bring a recreated resource back under management. RCA: who ran the apply, was the plan reviewed, was prevent_destroy set? Prevention: mandatory plan reviews, prevent_destroy on critical resources, restricted IAM.

Chapter 15 – Top 80 Interview Questions

Q: What is Terraform and how does it differ from Ansible?

A: Terraform is declarative IaC for provisioning infrastructure (creating EC2, VPC, RDS). Ansible is procedural configuration management (installing software, configuring OS). They complement each other: Terraform creates the infrastructure; Ansible configures it. Both can overlap but Terraform is the standard for cloud resource provisioning.

Q: What is idempotency in Terraform?

A: Running terraform apply multiple times on the same configuration produces the same result without unintended side effects. If resources already exist as desired, Terraform takes no action.

Q: What is HCL?

A: HashiCorp Configuration Language — a declarative language used to write Terraform configurations. Human-readable, JSON-compatible.

Q: Explain the Terraform dependency graph.

A: Terraform builds a Directed Acyclic Graph (DAG) of all resources and their dependencies before executing. Resources with no dependencies can be created in parallel. Resources that depend on others wait for their dependencies to complete. This graph determines the order of creation and destruction.

Q: What is a Terraform workspace?

A: Workspaces allow multiple state files within the same backend configuration. Useful for simple dev/staging/prod isolation where the code is identical but variable values differ. Not recommended for complex multi-environment setups — separate directories with separate backends are more maintainable.

Q: How do you import existing AWS resources into Terraform?

A: Run 'terraform import aws_instance.web i-0abc123'. This creates a state entry for the resource. Then write the matching resource block in config. Run terraform plan — drift will show attributes not yet in config. Add them. Test import in dev before production.

Q: What is terraform taint?

A: Deprecated in Terraform 1.0+. It marked a resource for forced replacement on next apply. Now use 'terraform apply -replace=aws_instance.web' instead — same effect without modifying state.

Q: What is the difference between terraform destroy and removing the resource from config?

A: Removing from config tells Terraform 'I no longer want to manage this.' On next apply, Terraform will destroy it. terraform destroy destroys everything in state. In production, prefer removing specific resources from config rather than running destroy to avoid accidents.

Q: How do you pass outputs between Terraform modules in different state files?

A: Use the `terraform_remote_state` data source. The producer module exposes outputs. The consumer reads the remote state bucket/key and accesses outputs. Alternatively, use SSM Parameter Store to share values without coupling via state.

Q: What is OpenTofu?

A: OpenTofu is an open-source fork of Terraform created after HashiCorp changed Terraform's license from MPL to BSL (Business Source License) in 2023. It is maintained by the Linux Foundation and is largely compatible with Terraform. Used by organizations that require a fully open-source IaC tool.

Q: What happens to state if you rename a resource in your config?

A: Terraform sees the old name removed and the new name as a new resource — it will destroy the old and create a new one. To prevent this: use `'terraform state mv old_name new_name'` to update state before applying the config rename.

Q: Why is the state file dangerous and how do you protect it?

A: State contains all resource attributes including sensitive values in plain text. Protect by: storing in S3 with SSE-KMS encryption, restricting access via IAM, enabling S3 versioning, enabling access logging, and never committing state to git.

Q: What is state locking and when does it fail?

A: State locking prevents concurrent terraform operations that could corrupt state. With DynamoDB locking, a lock is acquired at the start of plan/apply and released at the end. It fails if the previous operation was interrupted without releasing the lock — must be manually cleared from DynamoDB.

Q: How does terraform refresh work?

A: terraform refresh queries real infrastructure and updates state to match reality — without making any changes. In Terraform 1.x, refresh behavior is integrated into plan by default. Running it standalone is now considered legacy. Use `terraform plan -refresh-only` for explicit refresh.

Q: What is the Terraform Module Registry?

A: A public repository of community and official modules at registry.terraform.io. AWS maintains official modules for VPC, EKS, RDS, etc. Use with version pinning. Enterprises often mirror or maintain private registries for compliance.

Q: Can you pass a module's output to another module directly?

A: No. Module A outputs a value. The root module receives it via `module.A.output_name`. The root passes it as an input variable to Module B. Direct module-to-module referencing is not supported.

Q: What happens if two resources have circular depends_on?

A: Terraform will error: `'Cycle: resource_a, resource_b'`. You must break the cycle by removing one dependency or redesigning the resources. Circular dependencies indicate a design flaw.

Q: When should you use ignore_changes = all?

A: Almost never in production. It tells Terraform to ignore all attribute drift, making Terraform blind to real changes. Use specific attribute names instead. If you're tempted to use `'all'`, it's a sign the resource should not be managed by Terraform.

Q: What is the `for_each` limitation with lists?

A: `for_each` requires a map or set of strings — it cannot take a plain list directly. Convert with `toset(list)` or build a map. This is intentional: list indices are ambiguous (reordering changes what index maps to what resource).

Q: Can you use `count` and `for_each` on the same resource?

A: No. A resource block can use either `count` or `for_each`, never both simultaneously.

Q: How do you deploy Terraform across multiple AWS accounts?

A: Use provider aliases. Each alias assumes a role in the target account via `assume_role`. Pass different provider aliases to modules. Alternatively, separate Terraform workspaces/directories per account with account-specific backend configurations and IAM roles.

Q: How do you manage multi-region deployments?

A: Use provider aliases with different regions. Or separate Terraform projects per region with their own state files. Separate state per region prevents accidental cross-region destroy operations.

Q: How do you use SSM Parameter Store with Terraform?

A: Read parameters via data `'aws_ssm_parameter'`. Write parameters via resource `'aws_ssm_parameter'`. Use `SecureString` type with KMS for secrets. This decouples secrets management from Terraform code.

Q: Why save the plan file and apply it instead of re-planning?

A: Between plan review and apply, infrastructure can change (someone else applies, AWS changes, etc.). Saving the plan ensures you apply exactly what was reviewed. `terraform apply tfplan` applies the saved plan without re-evaluating.

Q: How do you prevent terraform apply in CI/CD without approval?

A: Use branch protection rules: plan runs on every PR, apply only runs on merge to main after required reviewer approvals. Atlantis requires explicit `'atlantis apply'` comment from an approver. Terraform Cloud has policy-based run approvals.

Q: How do you debug a Terraform error?

A: Set `TF_LOG=DEBUG` (or `TRACE` for maximum detail) and `TF_LOG_PATH=./terraform.log`. Review logs for API calls, error responses, and stack traces. Check provider documentation for the specific error. Common culprits: IAM permissions, API throttling, resource limits, dependency ordering.

Q: A terraform plan shows no changes but your AWS resource is different from config. Why?

A: The resource attribute might be managed by `ignore_changes`. Or the attribute might be computed (Terraform doesn't track it after creation). Or Terraform's provider version doesn't recognize that attribute. Run `terraform refresh` to update state and re-plan.

Chapter 16 – Cheat Sheets & One-Day Revision Guide

Core Command Cheat Sheet

Command	Purpose
terraform init	Initialize: download providers, configure backend
terraform validate	Check syntax and config validity
terraform fmt	Format code (use -check in CI to enforce)
terraform plan	Preview changes; use -out=tfplan to save
terraform apply	Apply changes; use tfplan to apply saved plan
terraform destroy	Destroy all managed resources
terraform state list	List all resources in state
terraform state show	Show state of a specific resource
terraform state mv	Rename resource in state (no AWS change)
terraform state rm	Remove from state (resource stays in AWS)
terraform import	Import existing AWS resource into state
terraform apply -replace=	Force replacement of a specific resource
terraform apply -refresh-only	Update state from real infra (no changes)
TF_LOG=DEBUG terraform plan	Enable debug logging

Key Concepts Quick Reference

State file	JSON map of Terraform config → real AWS resources. Never commit to git.
Backend	Where state is stored (S3) and locked (DynamoDB).
Provider	Plugin that calls AWS APIs. Pin version in required_providers.
Resource	Infrastructure object Terraform manages (creates/updates/destroys).
Data source	Read-only reference to existing infrastructure (not managed by this config).
Module	Reusable group of resources. Communicate via outputs through root.

Variable	Input to a module. Set via tfvars, env vars, or -var flag.
Local	Computed value derived within a module. Not settable externally.
Output	Exposed value from a module or displayed after apply.
Workspace	Separate state file within same backend. Use for simple isolation only.
count	Creates N identical resources. Uses index — removing middle item shifts indexes.
for_each	Creates named resources from map/set. Removing a key only affects that resource.
create_before_destroy	Creates replacement before destroying original. Reduces downtime.
prevent_destroy	Blocks terraform destroy on resource. Protect production critical infra.
ignore_changes	Ignores specific attribute drift. Use for externally managed attributes only.
depends_on	Explicit dependency. Use only when Terraform can't auto-detect it.
sensitive=true	Masks value in logs. Does NOT encrypt in state file.
terraform import	Bring existing AWS resource under Terraform management.
DynamoDB lock	Prevents concurrent applies. LockID column — String primary key.
S3 versioning	Enables state file recovery. Always enable on state bucket.

One-Day Revision Checklist

- ■ Explain Terraform workflow (init → plan → apply) including what happens internally
- ■ Explain why remote state is required in production and how S3+DynamoDB works
- ■ Explain what happens if state file is deleted and how to recover
- ■ Explain count vs for_each and when to use each
- ■ Explain create_before_destroy, prevent_destroy, ignore_changes with real use cases
- ■ Explain module structure and how modules communicate
- ■ Explain how to deploy Terraform in CI/CD without access keys
- ■ Explain how to handle secrets — never in .tf files, use Secrets Manager or SSM
- ■ Walk through 5 production troubleshooting scenarios confidently
- ■ Explain difference between resource and data source

- ■ Explain provider version pinning and .terraform.lock.hcl

Terraform Interview Handbook — Intermediate SRE / Cloud Operations Edition. Study smart, answer with production context, and explain the 'why' behind every decision.